

SQL Advanced - Cheat Sheet

The schema for every example is the same e-commerce model:

```
users(id, email, name, created_at)
products(id, name, price, category_id)
orders(id, user_id, status, total, created_at)
order_items(id, order_id, product_id, quantity, price)
```

Examples use Postgres syntax, with notes for MySQL and SQL Server where it matters.

Indexes deep dive

Indexes are how the database avoids reading every row. They're also the #1 reason interviewers ask "why is this query slow?" Get this section cold.

B-tree (the default)

Nearly every index you create is a B-tree. Postgres, MySQL (InnoDB), SQL Server, and Oracle all default to B-tree. They're balanced, ordered, and good for equality (=), range (< , > , BETWEEN), and prefix-match LIKE (LIKE 'foo%').

```
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_created_at ON orders(created_at);
```

Hash indexes

Postgres has hash indexes, but they only help with = . No range queries, no ordering. Honestly, just use B-tree. Hash indexes are a trivia question more than a real tool.

```
CREATE INDEX idx_users_email_hash ON users USING HASH (email);
```

Composite (multi-column) indexes

A composite index covers multiple columns in order. Column order matters a lot.

```
CREATE INDEX idx_orders_user_status ON orders(user_id, status);
```

This index helps queries that filter by user_id alone, or by user_id AND status . It does NOT help queries that filter by status alone. That's the leftmost-prefix rule. If you only ever query by status, you need a separate index on status .

Rule of thumb for column order: equality columns first (high selectivity), then range columns last.

Covering indexes / INCLUDE columns

Postgres 11+ supports INCLUDE columns. These add extra columns to the index leaf pages without making them part of the search key. This enables index-only scans.

```
CREATE INDEX idx_orders_user_inc ON orders(user_id) INCLUDE (status, total);
```

Now `SELECT status, total FROM orders WHERE user_id = 42` can be served entirely from the index. No heap fetch needed.

Partial indexes

A partial index covers only rows matching a WHERE clause. Smaller, faster, cheaper to maintain.

```
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';
```

Good fit for “active records” patterns where 90% of the table is archived or soft-deleted and you only query the live rows.

Unique indexes

Enforce uniqueness and provide a fast lookup at the same time.

```
CREATE UNIQUE INDEX idx_users_email ON users(email);
```

Note: a UNIQUE constraint creates a unique index under the hood. Same thing.

Expression / functional indexes

If you query with a function on a column, a normal index won't help. Build the index on the expression instead.

```
-- Bad: doesn't use idx_users_email
SELECT * FROM users WHERE LOWER(email) = 'rohit@example.com';

-- Fix: index the expression
CREATE INDEX idx_users_lower_email ON users(LOWER(email));
```

GIN and GiST (Postgres)

For non-scalar data like arrays, JSONB, full-text search, and geometry.

GIN (Generalized Inverted Index): good for “is X contained in Y” lookups. JSONB containment, array overlap, tsvector full-text.

GiST (Generalized Search Tree): good for ranges, geometry, and nearest-neighbor queries.

```
CREATE INDEX idx_orders_meta ON orders USING GIN (metadata jsonb_path_ops);
```

When indexes hurt

Every index slows down writes. Inserts, updates, and deletes all have to maintain every index on the table. On a write-heavy table with 12 indexes, expect a real penalty.

Indexes also cause bloat. Postgres B-tree indexes don't shrink automatically after bulk deletes. You may need `REINDEX CONCURRENTLY` periodically.

Index-only scans

If the index contains every column the query needs, the database skips the heap (table) entirely. Big speedup on wide tables. Use `INCLUDE` columns to make this work.

Quick rule of thumb

Index columns used in `WHERE`, `JOIN`, and `ORDER BY`. Skip indexes on low-cardinality columns (booleans, status fields with 3 values) unless you're pairing them with a partial index. Every index costs ~10-30% on write throughput, so don't carpet-bomb the table.

EXPLAIN and query plans

`EXPLAIN` shows you what the planner intends to do. `EXPLAIN ANALYZE` actually runs the query and shows real timings. Always reach for `ANALYZE` when debugging.

```
EXPLAIN ANALYZE
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE u.created_at > '2025-01-01'
GROUP BY u.id, u.name;
```

Reading a Postgres plan

A typical plan looks like this:

```

HashAggregate (cost=1245.32..1289.45 rows=4413 width=40) (actual time=12.453..15.221 ro
  Group Key: u.id
    -> Hash Right Join (cost=234.50..1123.20 rows=24424 width=36) (actual time=2.110..10
      Hash Cond: (o.user_id = u.id)
        -> Seq Scan on orders o (cost=0.00..789.00 rows=50000 width=8) (actual time=0.
        -> Hash (cost=189.30..189.30 rows=4413 width=32) (actual time=2.045..2.046 row
          -> Index Scan using idx_users_created_at on users u (cost=0.29..189.30 r
            Index Cond: (created_at > '2025-01-01'::date)
Planning Time: 0.342 ms
Execution Time: 15.789 ms

```

Read it from the deepest indented node outward. That's the order of execution.

Node types you'll see

- **Seq Scan:** read every row. Fine for small tables, scary for big ones.
- **Index Scan:** walk an index, then fetch matching rows from the heap.
- **Index Only Scan:** walk an index, skip the heap entirely. Best case.
- **Bitmap Heap Scan:** bitmap index reads many index entries, sorts by physical location, then fetches the heap. Better than Index Scan when many rows match.
- **Hash Join:** build a hash table on one side, probe with the other. Good for big-to-big joins.
- **Merge Join:** both sides pre-sorted, walk them in lockstep. Good when both sides are already ordered.
- **Nested Loop:** for each row in outer, lookup in inner. Cheap for small outer, deadly for large outer.

Cost numbers

`cost=234.50..1123.20` is `startup_cost..total_cost` in planner units. The number isn't milliseconds, it's an abstract estimate. Compare nodes against each other within the same plan, not across plans.

Rows: estimate vs actual

`rows=4413` is the planner's estimate. `actual rows=4200` is the truth. If estimate and actual diverge by 10x or more, your statistics are stale. Run `ANALYZE users` (or `VACUUM ANALYZE`) to refresh.

EXPLAIN with BUFFERS

```
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
```

Shows shared buffer hits vs disk reads. If you see `Buffers: shared read=10000`, you're hammering disk. If you see `shared hit=10000`, you're hitting cache. Cache hits are roughly 100x faster.

Sample annotation

For the plan above:

1. Innermost: index scan on `users.created_at` returns 4413 rows. Good, that index works.
2. Postgres builds a hash table from those 4413 user rows (Hash node).
3. Seq Scan on orders (50000 rows). The planner picked Seq Scan because we needed almost all orders anyway, an index scan would have been slower.
4. Hash Right Join probes the hash with each order. Result: 23890 joined rows.
5. HashAggregate groups by `u.id` and counts. Final: 4200 groups in ~16 ms.

If the orders table were 50 million rows instead of 50k, you'd want an index on `orders.user_id` and the planner would probably switch to a Nested Loop with Index Scan.

Query tuning techniques

Filter early

Push predicates as deep into the plan as possible. Filtering 1M rows down to 1k before a join is far cheaper than joining 1M rows and then filtering.

The planner usually does this for you, but only if your predicates are sargable (see below).

Avoid SELECT *

Three problems. You fetch columns you don't need (network and memory waste). You break index-only scans (the index doesn't have all the columns). Schema changes can silently break your code (column added, suddenly your DTO mapper barfs).

Be explicit. List the columns.

Avoid functions on indexed columns in WHERE

```
-- Bad: idx_users_email is useless
WHERE LOWER(email) = 'rohit@example.com'

-- Bad: idx_orders_created_at is useless
WHERE DATE(created_at) = '2025-05-23'

-- Good: rewrite to leave the column alone
WHERE created_at >= '2025-05-23' AND created_at < '2025-05-24'
```

If you really need a function, build a functional index.

Avoid leading wildcards in LIKE

```
-- Can't use the index
WHERE name LIKE '%shukla'

-- Can use the index (prefix match)
WHERE name LIKE 'shukla%'
```

For leading-wildcard search, you need a trigram index (`pg_trgm` extension in Postgres) or full-text search.

OR vs UNION ALL

Sometimes the planner can't use indexes well with `OR` :

```
-- Hard to plan: may seq scan
SELECT * FROM orders WHERE user_id = 42 OR status = 'pending';

-- Easier to plan: each branch uses its own index
SELECT * FROM orders WHERE user_id = 42
UNION ALL
SELECT * FROM orders WHERE status = 'pending' AND user_id != 42;
```

Not always faster. Always check with `EXPLAIN ANALYZE`. Modern planners are smart about `OR`, but legacy ones aren't.

Sargable predicates

“Sargable” means “Search ARGument ABLE”, a predicate the index can use. A predicate is sargable when the indexed column is on one side, untouched by functions, with a constant or computable expression on the other.

Predicate	Sargable?
WHERE id = 42	Yes
WHERE id BETWEEN 10 AND 20	Yes
WHERE LOWER(name) = 'foo'	No
WHERE id + 1 = 43	No
WHERE name LIKE 'a%'	Yes
WHERE name LIKE '%a'	No

When to denormalize

Normalize first, denormalize when measured pain demands it. Common cases:

- Hot-path aggregates (cache `order_count` on `users`).
- Reporting dashboards (precompute via materialized views or ETL).
- Wide-fan-out reads (embed JSON for read-mostly data).

The trade-off is write complexity and staleness. Be deliberate.

Statistics: ANALYZE

The planner makes decisions based on table statistics: row counts, value distributions, null fractions. Postgres auto-runs `ANALYZE` via autovacuum, but bulk inserts can outpace it. After a big load:

```
ANALYZE orders;  
-- or for everything  
VACUUM ANALYZE;
```

If `EXPLAIN ANALYZE` shows estimates wildly different from actuals, suspect stale stats first.

LIMIT + ORDER BY: the index trick

```
SELECT * FROM orders  
WHERE user_id = 42  
ORDER BY created_at DESC  
LIMIT 20;
```

If you have an index on `(user_id, created_at DESC)`, Postgres can walk the index and stop after 20 rows. Big speedup vs sorting all of the user's orders.

Common antipatterns

N+1 queries

The classic Hibernate disaster. You load 100 users, then iterate and fetch each user's orders. That's $1 + 100 = 101$ queries. Should be 1 or 2.

```
// N+1 disaster  
List<User> users = userRepo.findAll();  
for (User u : users) {  
    u.getOrders().forEach(System.out::println); // lazy load per user  
}  
  
// Fix with JOIN FETCH  
@Query("SELECT u FROM User u LEFT JOIN FETCH u.orders")  
List<User> findAllWithOrders();
```

How to spot in tests: `log SQL (spring.jpa.show-sql=true plus org.hibernate.SQL=DEBUG)`, or count queries with a Hibernate Statistics object.

SELECT * everywhere

Already covered. The fix is "list your columns."

OFFSET on large pagination

`OFFSET 1000000 LIMIT 20` makes the database scan and discard 1 million rows just to throw them away. Brutal on big tables.

Use keyset or cursor pagination instead:

```
-- First page
SELECT * FROM orders WHERE user_id = 42
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- Next page (pass the last row's created_at and id)
SELECT * FROM orders WHERE user_id = 42
AND (created_at, id) < ('2025-05-23 10:00:00', 1234)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Constant time regardless of page depth.

Implicit type coercion

```
-- user_id is BIGINT but you bind a String
WHERE user_id = '42'
```

Postgres casts the string to bigint, fine. But MySQL with a string column and an int comparison can convert the column (not the literal) and disable the index. Always match types.

Functions on indexed columns

Covered above. Don't wrap indexed columns in functions.

DISTINCT to “fix” a bad join

If you have duplicates in a join result, the join is wrong. Adding `DISTINCT` masks the bug and forces a sort or hash. Fix the join condition instead, usually you missed a join key or you need a semi-join (`EXISTS`).

```
-- Bad: hides the join duplication
SELECT DISTINCT u.* FROM users u
JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid';

-- Good: semi-join, no dedup needed
SELECT u.* FROM users u
WHERE EXISTS (
  SELECT 1 FROM orders o WHERE o.user_id = u.id AND o.status = 'paid'
);
```


Transactions and ACID

ACID is the contract every relational database promises:

- **Atomicity:** all statements in a transaction commit together or none of them do.
- **Consistency:** constraints (FKs, uniques, checks) hold at commit time.
- **Isolation:** concurrent transactions don't see each other's half-done work (degree depends on isolation level).
- **Durability:** once committed, the data survives crashes (WAL/redo log on disk).

BEGIN / COMMIT / ROLLBACK

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;  
-- or ROLLBACK; to undo
```

SAVEPOINT

Nested rollback points within a transaction.

```
BEGIN;  
INSERT INTO orders(...) VALUES (...);  
SAVEPOINT after_order;  
INSERT INTO order_items(...) VALUES (...); -- might fail  
-- if it failed:  
ROLLBACK TO after_order;  
-- order is still there, only the item insert is undone  
COMMIT;
```

Transaction scope in code

Rule: never hold a transaction open across user input or external I/O. The connection is locked, the transaction holds row versions back from vacuum, and you're one slow API call away from a connection pool meltdown.

In Java/Spring:

```
// Bad: transaction wraps an HTTP call to Stripe  
@Transactional  
public void chargeAndOrder(Order o) {  
    paymentClient.charge(o.getAmount()); // 2-second HTTP call  
    orderRepo.save(o);  
}
```

The transaction should wrap only the database work. Charge first (or use an outbox/saga pattern), then transactionally write.

Isolation levels

Four standard levels. The higher you go, the fewer anomalies you see, the more locking and contention you get.

The phenomena

- **Dirty Read:** read another transaction's uncommitted data.
- **Non-Repeatable Read:** read row X, another transaction commits an UPDATE to X, re-read row X, get different data.
- **Phantom Read:** re-run a range query, get different rows because another transaction inserted matches.
- **Lost Update:** two transactions read a value, both update it, one update is silently overwritten.
- **Write Skew:** two transactions read overlapping sets, each writes based on its read, both commit, but the combined state violates an invariant.

The table

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Write Skew
READ UNCOMMITTED	Possible	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible (SQL std) / Prevented (Postgres)	Possible
SERIALIZABLE	Prevented	Prevented	Prevented	Prevented

Defaults

- Postgres: `READ COMMITTED` .
- MySQL InnoDB: `REPEATABLE READ` .
- SQL Server: `READ COMMITTED` .
- Oracle: `READ COMMITTED` .

Snapshot isolation (Postgres REPEATABLE READ)

Postgres REPEATABLE READ uses MVCC snapshots. Every read in the transaction sees the same snapshot from transaction start. Phantom reads can't happen because new rows aren't visible to the snapshot. Write skew still can.

SERIALIZABLE in Postgres (SSI)

Postgres SERIALIZABLE uses Serializable Snapshot Isolation (SSI). It tracks read/write dependencies between concurrent transactions. If it detects a dangerous pattern, it aborts one with `ERROR: could not serialize access due to read/write dependencies`. You retry.

The trade-off: less locking than traditional serializable, but you must be ready to retry transactions in application code.

Setting isolation

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
-- or  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

In Spring: `@Transactional(isolation = Isolation.SERIALIZABLE)`.

Locking

Row vs table locks

Most modern engines (Postgres, InnoDB) lock at the row level for normal DML. Table-level locks happen during DDL (`ALTER TABLE`), `LOCK TABLE` statements, or `VACUUM FULL` .

Shared vs exclusive

- **Shared (S)**: readers can hold; multiple readers can share.
- **Exclusive (X)**: writers hold; blocks everyone else.

SELECT FOR UPDATE

Locks the rows for update until commit. Other writers and other FOR UPDATE readers block. Plain readers don't block (in Postgres MVCC).

```
BEGIN;  
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;  
-- nobody else can lock or modify this row  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
COMMIT;
```

SELECT FOR UPDATE SKIP LOCKED

Postgres 9.5+. Skip rows that another transaction has locked. Good fit for queue/job-worker patterns.

```
SELECT id, payload FROM jobs
WHERE status = 'pending'
ORDER BY created_at
LIMIT 1
FOR UPDATE SKIP LOCKED;
```

Multiple workers pull different jobs without blocking each other.

SELECT FOR UPDATE NOWAIT

Fail immediately if a row is locked, instead of waiting.

```
SELECT * FROM accounts WHERE id = 1 FOR UPDATE NOWAIT;
-- ERROR: could not obtain lock on row in relation "accounts"
```

Advisory locks (Postgres)

Application-defined locks, not tied to any row. Use them for cross-process coordination (e.g., “only one worker runs this cron job at a time”).

```
SELECT pg_try_advisory_lock(12345); -- returns true if acquired
-- ... do work ...
SELECT pg_advisory_unlock(12345);
```

Deadlocks

Two transactions each hold a lock the other wants. The database detects the cycle and aborts one with `ERROR: deadlock detected`. The classic case:

```
T1: UPDATE row 1; UPDATE row 2;
T2: UPDATE row 2; UPDATE row 1;
```

Prevention: always lock rows in the same order. In application code, wrap in a retry:

```
@Retryable(value = DeadlockLoserDataAccessException.class, maxAttempts = 3)
@Transactional
public void transfer(long from, long to, BigDecimal amount) { ... }
```

Lock escalation

SQL Server escalates many row locks to a table lock when it hits a threshold (default 5000 locks). Postgres doesn't escalate, ever. This is one reason Postgres handles huge concurrent workloads

gracefully but uses more memory for lock tracking.

Optimistic vs pessimistic locking

Pessimistic: lock the row up front with `SELECT FOR UPDATE`. Safe but contention-heavy.

Optimistic: don't lock. Add a version column. On update, check the version matches, increment it; if not, the row changed under you, retry.

```
-- Read
SELECT id, balance, version FROM accounts WHERE id = 1;
-- (in app: do work, prepare new balance)

-- Write
UPDATE accounts SET balance = ?, version = version + 1
WHERE id = 1 AND version = ?;
-- if 0 rows updated, someone else won the race; retry
```

In JPA: `@Version` on a field, Hibernate generates the WHERE-with-version clause and throws `OptimisticLockException` on miss. Good default for low-contention scenarios.

MVCC (Postgres-flavored)

Multi-Version Concurrency Control. The core trick: instead of overwriting rows in place, every UPDATE writes a new version of the tuple. Each version has visibility info (`xmin` = inserting txn, `xmax` = deleting txn). Readers see whichever version is “visible” under their transaction snapshot.

Consequences:

- Readers don't block writers, writers don't block readers. A `SELECT` never has to wait for an `UPDATE`. Big win for concurrent workloads.
- Old row versions accumulate. When all transactions that could see an old version are done, it becomes “dead”. `VACUUM` cleans them up.
- Long-running transactions hold back vacuum. If you start a transaction at noon and leave it open until 5pm, no rows changed between noon and 5pm can be vacuumed until your transaction ends. This is how you get table bloat.

Bloat

If vacuum can't keep up (or is held back by long transactions), tables and indexes grow with dead tuples. Query performance suffers because more pages have to be read for the same live rows.

Monitor with `pg_stat_user_tables.n_dead_tup`. Fix with `VACUUM` (regular), `VACUUM FULL` (rewrites the table, takes an exclusive lock), or `pg_repack` (online rewrite, no exclusive lock).

Practical rules

- Don't hold transactions open longer than you need.

- Don't BEGIN a transaction in your app and then call out to Stripe.
- Monitor `pg_stat_activity` for `state = 'idle in transaction'` . Kill old ones.
- Autovacuum is your friend. Tune it, don't disable it.

Partitioning

For tables that grow without bound (events, logs, orders by date), partitioning splits the data physically by a key. The planner can prune partitions and only touch the relevant ones.

Why

- Cheap archival: dropping a partition is constant-time (vs DELETE scanning millions of rows).
- Smaller indexes per partition: each partition has its own index. Smaller indexes mean faster lookups.
- Parallel scans across partitions.

Range partitioning (most common)

```
CREATE TABLE orders (  
  id BIGSERIAL,  
  user_id BIGINT,  
  created_at TIMESTAMPTZ NOT NULL,  
  total NUMERIC,  
  PRIMARY KEY (id, created_at)  
) PARTITION BY RANGE (created_at);  
  
CREATE TABLE orders_2025_q1 PARTITION OF orders  
  FOR VALUES FROM ('2025-01-01') TO ('2025-04-01');  
  
CREATE TABLE orders_2025_q2 PARTITION OF orders  
  FOR VALUES FROM ('2025-04-01') TO ('2025-07-01');
```

`SELECT ... WHERE created_at >= '2025-05-01'` only scans `orders_2025_q2` . That's partition pruning.

List and hash

List: partition by enum-like values. `PARTITION BY LIST (region)` with separate partitions for `'us'` , `'eu'` , etc.

Hash: partition by hash of a key, for even distribution. `PARTITION BY HASH (user_id)` with N partitions. Used when no natural range works.

Trade-offs

- Indexes must include the partition key (or be defined per partition).
- Foreign keys referencing a partitioned table got better in PG12 but historically were annoying.

- Cross-partition queries don't get pruned and run against all partitions.
- More complexity. Don't partition until the table is big enough to matter (rule of thumb: 100M+ rows or 100GB+).

Replication (brief)

Primary / replica

The primary handles writes. Replicas stream changes from the primary's WAL (write-ahead log) and serve reads. You scale reads horizontally by adding replicas.

Sync vs async

Async (default): primary commits locally, then ships WAL. Fast, but a primary crash can lose recent writes that haven't replicated.

Sync: primary waits for at least one replica to confirm before committing. Slower, no data loss.

Postgres: `synchronous_commit = on` plus `synchronous_standby_names` .

Replication lag

The delay between a write on the primary and its visibility on a replica. If your app reads its own writes from a replica, you'll see "I just inserted this and now it's gone" bugs. Fix: read your own writes from the primary, route eventual-consistency reads to replicas.

Failover

When the primary dies, promote a replica to primary. Tools: Patroni, repmgr, pg_auto_failover, managed services (RDS, Aurora, Cloud SQL). The hard part isn't promotion, it's making sure no two nodes both think they're primary (split brain).

Logical vs physical (Postgres)

Physical replication: byte-for-byte WAL streaming. Replica is identical to primary. Used for HA and read replicas.

Logical replication: row-level changes serialized as logical events. Cross-version, partial table replication, change data capture (CDC) feeding into Kafka. Tools: Debezium uses logical decoding.

Stored procedures, functions, triggers

When to use

- Performance-critical batch logic that needs many round-trips collapsed into one.
- Complex constraints that can't be expressed declaratively.
- Audit trails (triggers).

When NOT to use

Business logic. Keep that in the app. SQL-side logic is hard to version, test, and debug. Your team uses Git for Java code, not for PL/pgSQL stored in a database.

PL/pgSQL function example

```
CREATE OR REPLACE FUNCTION transfer(  
    from_id BIGINT,  
    to_id BIGINT,  
    amount NUMERIC  
) RETURNS VOID AS $$  
DECLARE  
    sender_balance NUMERIC;  
BEGIN  
    SELECT balance INTO sender_balance  
    FROM accounts WHERE id = from_id FOR UPDATE;  
  
    IF sender_balance < amount THEN  
        RAISE EXCEPTION 'Insufficient funds';  
    END IF;  
  
    UPDATE accounts SET balance = balance - amount WHERE id = from_id;  
    UPDATE accounts SET balance = balance + amount WHERE id = to_id;  
END;  
$$ LANGUAGE plpgsql;
```

Call it: `SELECT transfer(1, 2, 100);`

Triggers and hidden actions

```
CREATE OR REPLACE FUNCTION audit_order_changes()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO order_audit(order_id, old_status, new_status, changed_at)  
    VALUES (OLD.id, OLD.status, NEW.status, NOW());  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_audit_orders  
AFTER UPDATE ON orders  
FOR EACH ROW  
WHEN (OLD.status IS DISTINCT FROM NEW.status)  
EXECUTE FUNCTION audit_order_changes();
```

The hidden-action problem: a new developer joins, runs `UPDATE orders SET status = 'shipped' WHERE id = 5`, and has no idea that an audit row was just written. Triggers fire invisibly. They make debugging hard. Use them sparingly, document them in the schema repo.

JSON / JSONB

JSON vs JSONB

- **JSON** : stored as text. Preserves whitespace and key order. Reparsed on every read.
- **JSONB** : binary format. Indexable, faster operators, key order not preserved.

Use **JSONB** . Always.

Operators

```
-- -> returns JSONB (subdocument)
SELECT metadata -> 'address' FROM users;

-- ->> returns TEXT (extracted scalar)
SELECT metadata ->> 'email' FROM users;

-- @> contains
SELECT * FROM orders WHERE metadata @> '{"source": "mobile"}';

-- ? key exists
SELECT * FROM orders WHERE metadata ? 'discount_code';

-- #> and #>> for paths
SELECT metadata #>> '{address,city}' FROM users;
```

GIN index on JSONB

```
CREATE INDEX idx_orders_metadata ON orders USING GIN (metadata);
-- or smaller/faster for @> only
CREATE INDEX idx_orders_metadata ON orders USING GIN (metadata jsonb_path_ops);
```

Now `WHERE metadata @> '{"source": "mobile"}'` uses the index.

When JSON beats relational

- Schemaless or sparse attributes (every product has different specs).
- Event payloads or metadata you don't query relationally.
- Read-mostly nested data (config blobs, feature flags per user).

When it doesn't

- You need joins on the data. JSONB foreign keys aren't a thing.
- You need strict types and constraints.
- You need normal indexing patterns (B-tree on a column). JSON paths are doable but more work.

If you find yourself doing `WHERE metadata->>'user_id' = '42'` a lot, that field belongs in a column.

Materialized views

A view that's actually stored. The query runs once, results are cached on disk, future reads of the view are dirt cheap. The data goes stale until you refresh.

```
CREATE MATERIALIZED VIEW user_order_summary AS
SELECT u.id, u.email, COUNT(o.id) AS order_count, SUM(o.total) AS total_spend
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.email;

CREATE UNIQUE INDEX idx_uos_id ON user_order_summary(id);
```

Refresh

```
-- Locks the view for reads while refreshing
REFRESH MATERIALIZED VIEW user_order_summary;

-- No readlock, but requires a unique index on the view
REFRESH MATERIALIZED VIEW CONCURRENTLY user_order_summary;
```

CONCURRENTLY is slower (it does a diff) but doesn't block readers. Always prefer it for production.

Use cases

- Precomputed dashboards (top sellers, daily revenue).
- Expensive aggregations refreshed nightly.
- Reporting tables sitting behind a BI tool.

Not for real-time data. The view is as fresh as your last refresh.

Java / JPA gotchas

N+1 problem

Already covered, but worth its own subsection because it's the #1 production performance bug in Spring/Hibernate apps.

Detection:

- Enable SQL logging: `spring.jpa.show-sql=true`,
`logging.level.org.hibernate.SQL=DEBUG`.
- Use `org.hibernate.stat.Statistics` to count queries per request.
- Tools: Hypersistence Optimizer, Datasource Proxy.

Fixes:

```
// 1. JOIN FETCH
@Query("SELECT u FROM User u LEFT JOIN FETCH u.orders WHERE u.id IN :ids")
List<User> findWithOrders(@Param("ids") List<Long> ids);

// 2. @EntityGraph
@EntityGraph(attributePaths = {"orders", "orders.items"})
List<User> findByIdIn(List<Long> ids);

// 3. Batch fetching (Hibernate-specific)
@BatchSize(size = 50)
@OneToMany(mappedBy = "user")
private List<Order> orders;
```

LazyInitializationException

You return an entity from a `@Transactional` service method. The transaction (and Hibernate session) closes. The controller tries to serialize the entity's lazy collection. Boom.

Fixes:

- Don't return entities. Return DTOs assembled inside the transaction.
- Use JOIN FETCH or `@EntityGraph` to eagerly load what you'll access.
- Avoid Open Session In View (it's enabled by default in Spring Boot, and it hides bugs).

@Transactional and connection pool exhaustion

A transactional method holds a connection from the pool for its entire duration. If you call external HTTP services inside `@Transactional`, you tie up the connection for seconds. 20 concurrent calls and your pool of 10 connections is gone.

Rule: keep `@Transactional` blocks tight. Do external I/O before or after, never inside.

Hibernate gets the SQL surprising

- **Cartesian explosion with multiple JOIN FETCH:** fetching two unrelated collections in one query multiplies rows. Limit to one collection per fetch query, or use `@EntityGraph` + multiple roundtrips.
- **Pagination + JOIN FETCH:** Hibernate logs `HHH000104: firstResult/maxResults specified with collection fetch; applying in memory!` It fetches everything and paginates in app memory. Disaster on large tables. Paginate IDs first, then fetch with JOIN.
- **@OneToMany without mappedBy:** Hibernate generates a join table you didn't ask for.
- **Cascade ALL with orphanRemoval:** forget you set it, suddenly removing an item from a list deletes it from the database.

Best practices

- Index by query pattern, not by intuition. Watch `pg_stat_statements` for slow queries and index those columns.
- Run EXPLAIN ANALYZE before and after every change. Don't trust your gut.
- Keep transactions short. No external I/O inside `@Transactional`.
- Use connection pooling (HikariCP, PgBouncer). Don't open raw connections per request.
- Use prepared statements (parameterized queries). Stops SQL injection and lets the planner cache plans.
- Default to READ COMMITTED isolation. Step up to REPEATABLE READ or SERIALIZABLE only when you actually need it, and be ready to retry.
- Add a version column for optimistic locking on hot rows.
- Run ANALYZE after bulk loads. Stale stats lead to bad plans.
- Monitor replication lag and autovacuum activity in production.
- Use `SELECT FOR UPDATE SKIP LOCKED` for job queues. Don't poll with `SELECT WHERE status = 'pending'` and hope.
- Partition tables only when they're actually big. Premature partitioning is a real anti-pattern.
- Keep business logic in the app. Triggers and stored procedures are a debugging tax.
- Use `JSONB` (not `JSON`). And don't put fields in JSONB that you'll query relationally.
- Write database migrations in your repo, version-controlled (Flyway, Liquibase). Never edit the schema by hand in prod.

Common gotchas

- `NULL = NULL` is `NULL`, not `TRUE`. Use `IS NULL`, `IS DISTINCT FROM`.
- `COUNT(*)` includes NULLs; `COUNT(column)` doesn't.
- OFFSET on large pagination is O(n). Use keyset pagination.
- `LIKE '%foo'` can't use a normal B-tree index. Needs trigram or full-text.
- Wrapping an indexed column in a function (`LOWER(email)`) defeats the index unless it's a functional index.
- Hibernate's Open Session In View hides N+1 bugs in development and exposes them in production.
- `@Transactional` on a private method does nothing. Spring proxies only intercept public methods.
- `@Transactional(propagation = REQUIRES_NEW)` opens a new transaction and a new connection. Easy way to exhaust the pool.
- Postgres REPEATABLE READ prevents phantoms (snapshot isolation), but the SQL standard doesn't require that.
- SERIALIZABLE in Postgres can abort your transaction. Always retry on 40001 SQLSTATE.
- `VACUUM FULL` takes an exclusive lock and rewrites the table. Don't run it in business hours. Use `pg_repack` or schedule downtime.

- A long-running transaction (even one that just reads) holds back vacuum and causes bloat.
- EXPLAIN without ANALYZE shows estimates only. They can be wrong. Always run ANALYZE to see reality.
- Index Scan isn't always faster than Seq Scan . For a query that returns 80% of the table, Seq Scan wins.
- Foreign keys without indexes on the referencing column slow down deletes on the parent table.
- MySQL InnoDB and Postgres handle REPEATABLE READ differently. Don't assume cross-database behavior.
- Replicas can be behind. Reads from replicas after a write may not see the write. Read your own writes from the primary.

Quick reference

Topic	Key item	Notes
Default index	B-tree	Equality, range, prefix LIKE
Composite index	Order matters	Equality cols first, range cols last
Index-only scan	Use INCLUDE	All needed columns in the index
Partial index	WHERE on index	Smaller, faster for skewed data
Functional index	<code>ON (LOWER(email))</code>	When you query with functions
GIN	JSONB, arrays, fts	Use <code>jsonb_path_ops</code> for <code>@></code> only
EXPLAIN ANALYZE	Runs the query	Shows actual times and row counts
Buffers	<code>EXPLAIN (ANALYZE, BUFFERS)</code>	shared hit = cache, read = disk
Stale stats	Run ANALYZE	When estimate vs actual diverges >10x
Postgres default isolation	READ COMMITTED	
MySQL InnoDB default	REPEATABLE READ	
Prevent phantoms (PG)	REPEATABLE READ or SERIALIZABLE	PG REPEATABLE READ is snapshot iso
Prevent write skew	SERIALIZABLE only	PG uses SSI, may abort and retry
Row lock	<code>SELECT ... FOR UPDATE</code>	Blocks other writers
Queue worker	<code>FOR UPDATE SKIP LOCKED</code>	Workers don't block each other
Optimistic locking	<code>@Version</code> column	Retry on OptimisticLockException
Deadlock	DB aborts one txn	Retry on SQLSTATE 40P01
MVCC vacuum blocker	Long-running txn	Kill idle in transaction sessions

Topic	Key item	Notes
Partition key	RANGE on date	Drop old partition = O(1) archival
Read replica lag	Async by default	Read-your-writes from primary
Logical replication	CDC into Kafka	Debezium uses logical decoding
JSON path operator	<code>--></code> returns text	<code>-></code> returns JSONB
JSONB containment	<code>@></code>	GIN-indexable
Materialized view refresh	<code>REFRESH ... CONCURRENTLY</code>	Needs unique index
N+1 fix	JOIN FETCH / @EntityGraph	Detect with SQL logging
Lazy loading exception	Close to view boundary	Return DTOs, not entities
Keyset pagination	<code>WHERE (col, id) < (?, ?)</code>	Constant time per page
Sargable predicate	Indexed col, no function	<code>col = ?</code> , not <code>f(col) = ?</code>
SELECT *	Don't	Breaks index-only scans
Transaction + HTTP	Don't	Connection pool exhaustion